DISSENY D'UN SISTEMA DE MESURA DE FATIGA MUSCULAR MITJANÇANT NIRS

Desenvolupament de projectes d'electrònica

Enginyeria biomèdica

Lluís Francesc Collell i Maria del Mar Steinhauer

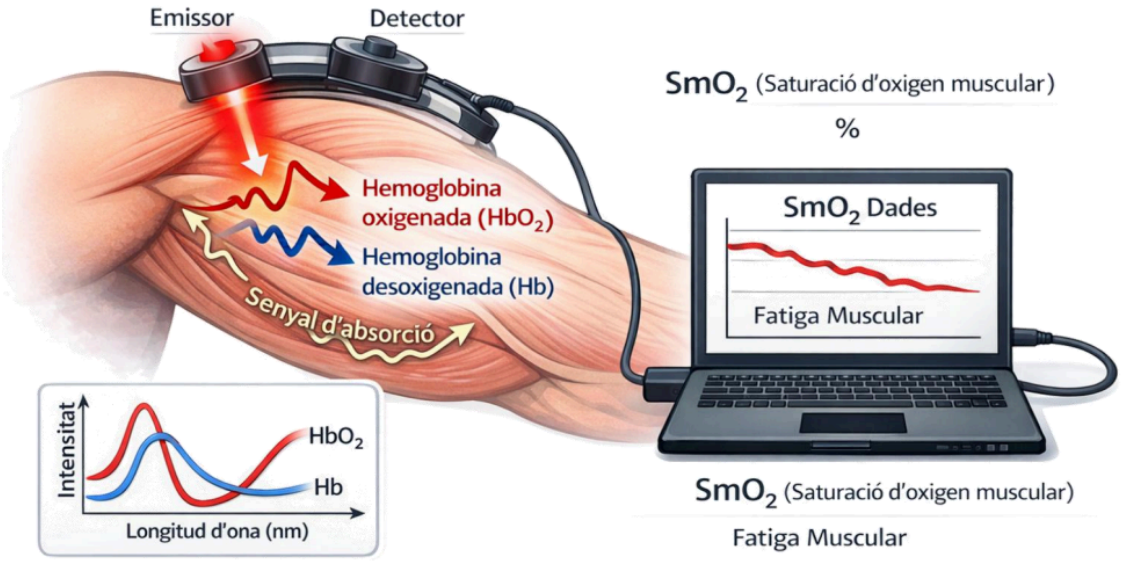

Fig. 1: Exemple de funcionament d'un sistema de mesura de fatiga muscular mitjançant NIRS. Descripció del projecte, DPE 2025-2026.

Funcionalitat del Sistema i Comunicació

El programari s'ha dissenyat sota un concepte híbrid que permet el treball amb hardware real o en mode simulat mitjançant la variable global **'g_isSimulation'.** Aquesta funcionalitat és clau per al desenvolupament, ja que permet validar la lògica matemàtica sense dependre de la placa física. La comunicació amb l'exterior es realitza mitjançant el protocol **Bluetooth Low Energy (BLE)**, utilitzant la llibreria NimBLE per garantir un consum de recursos mínim. La gestió interna de les dades es basa en el sistema operatiu **FreeRTOS**, on fem servir **cues (Queues)** per enviar la informació bruta des de la tasca d'adquisició fins a la de processament de forma asíncrona, evitant així qualsevol bloqueig en la presa de mostres.

Detalls de Programació i Optimització de Còmput

Programació de l'ADC: Configurat a **1000 SPS** (1 ms de conversió). Utilitzem el pin **'DRDY'** amb interrupcions físiques (**'ISR'**) i un semàfor binari per llegir la dada exactament quan està llesta, evitant el **'polling'** i alliberant CPU.

Temps i Sincronia: Cada subcicle dura **2 ms** (8 ms el total). La funció **'vTaskDelayUntil'** garanteix la periodicitat exacta en compensar automàticament el temps d'execució del codi, eliminant qualsevol retard acumulat.

Eficiència en la Transmissió: Enviem el valor de saturació com un enter de 16 bits (**'uint16_t'**) multiplicat per 100 (ex: **6542** per un **65.42%**). Això optimitza el Bluetooth, i l'script de Python recupera els decimals dividint per **100.0**.

Distribució per nuclis (Cores): L'adquisició crítica corre al **Core 1**, mentre que el processament i el BLE es gestionen al **Core 0**. Aquesta separació evita que l'activitat de ràdio del Bluetooth interfereixi en la precisió de la captura de dades.

Interfície Python i Registre de Dades

La interfície en Python s'ha dissenyat per ser una eina de monitorització robusta. Utilitza la llibreria **'bleak'** per connectar-se de forma asíncrona al dispositiu. La visualització es realitza amb **matplotlib** aplicant l'estil **'seaborn-v0_8-darkgrid'** per a una estètica professional. El sistema no només mostra la gràfica en temps real, sinó que genera un fitxer **'.csv'** amb marques de temps (**'timestamps'**). Aquesta solució és òptima perquè garanteix la persistència de les dades per a estudis posteriors i permet un seguiment precís de la sessió de mesura.

Arquitectura, Modularitat i Escalabilitat

El codi s'ha dividit en mòduls independents per maximitzar l'entenibilitat i facilitar l'escalabilitat del projecte. Aquesta separació de conceptes permet modificar el hardware sense alterar la lògica de càlcul:

'config.h': Centralitza tota la configuració del sistema, incloent-hi pins, constants físiques, UUIDs del Bluetooth i el selector de mode **'g_isSimulation'**.

'main.cpp': Inicialitza els perifèrics, crea les cues i semàfors de FreeRTOS i llança les tasques als dos nuclis de l'ESP32.

'sensor_hal': Capa d'abstracció que actua com a selector. Decideix si les dades es llegeixen del sensor real o del motor de simulació de forma transparent per a la resta del codi.

'adc_hw': Controlador específic del xip **'AD7172'**. Gestiona la comunicació per bus SPI i la configuració de registres per a la lectura de 24 bits.

'adc_sim': Generador de dades sintètiques que emula la resposta de l'hemoglobina i el soroll ambiental, permetent testar el sistema sense hardware.

'nirs_logic': Conté l'algorisme de Beer-Lambert. Transforma les intensitats lumíniques en valors de **'SmO2'** mitjançant càlculs matricials optimitzats.

'ble_service': Gestiona la comunicació inalàmbrica. S'encarrega de l'advertising i de l'enviament de notificacions asíncrones al client connectat.

'tasks': Defineix el comportament temporal del firmware. Controla els cicles d'adquisició, el processament de les dades i la freqüència de transmissió Bluetooth.

Càlcul de SmO2 i Constants Físiques

Per obtenir la saturació d'oxigen muscular, el programari aplica la Llei de Beer-Lambert Modificada. Aquest càlcul depèn de dos factors crítics: les constants d'extinció de l'hemoglobina i el valor de referència **'I0'**.

El paràmetre **'I0'** representa l'intensitat de llum inicial, és a dir, la quantitat de llum que rebria el sensor si no hi hagués cap tipus d'absorció pel camí. En el nostre disseny, hem agafat com a referència el valor màxim que pot registrar l'ADC de 24 bits, que és **'16.777.215'**. Tot i que no podem confirmar amb total seguretat si aquest valor és el més correcte des d'un punt de vista físic absolut (ja que requeriria un calibratge professional amb materials de referència), l'hem triat perquè ens serveix com a fons d'escala per detectar correctament les tendències i els canvis relatius en l'oxigenació.

Les constants d'extinció molar (**'EPS_O2'** i **'EPS_HB'**) defineixen quanta llum absorbeix l'hemoglobina segons la longitud d'ona (vermell o infraroig). L'algorisme utilitza aquests valors fixos per resoldre el sistema d'equacions i calcular el percentatge final de **'SmO2'**.